UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIA DA COMPUTAÇÃO

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina como requisito para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Antônio Augusto Fröhlich, Phd.
Coorientador: Matheus Wagner, Me.

Florianópolis
2026

Davi Menegaz Junkes

Gateway Semântico para Tradução de Mensagens ROS 2 para o Formato SmartData

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo curso de Graduação em Ciência da Computação.

Florianópolis, _____ de _____ de 2026.

Banca Examinadora:

Prof. Antônio Augusto Fröhlich, Phd.
Orientador
Universidade Federal de Santa Catarina

Matheus Wagner, Me.
Coorientador
Universidade Federal de Santa Catarina

José Luis Conradi Hoffmann, Phd.
Avaliador
Universidade Federal de Santa Catarina

RESUMO

A convergência entre robótica móvel e Internet das Coisas é central para a implementação dos Sistemas Ciber-Físicos. O *Robot Operating System 2* (ROS 2) consolidou-se como o padrão no desenvolvimento robótico e adotou o *Data Distribution Service* (DDS) como *middleware* de comunicação, garantindo entrega eficiente de mensagens. As mensagens nativas do ROS, porém, transmitem dados brutos de sensores sem o contexto físico e temporal necessário para sistemas de tomada de decisão autônoma. O conceito de *SmartData*, proposto por Fröhlich (2018), responde a essa lacuna ao encapsular cada leitura sensorial em um envelope que carrega, inseparavelmente do valor, sua unidade no Sistema Internacional, a origem espaço-temporal do dado e seu prazo de validade. Mais do que um formato, o SmartData define uma dinâmica de transdutores: produtores anunciados na rede respondem aos interesses de consumidores remotos, a cada atualização ou periodicamente, e atuadores são comandados pela escrita de um novo valor. Este trabalho projeta, implementa e avalia um *Gateway* semântico que exporta sensores e atuadores ROS 2 como transdutores SmartData e importa mensagens no formato SmartData para redes ROS 2. Um descritor YAML enumera os tópicos ROS e suas regras de decomposição, e um gerador em Python emite, em tempo de compilação, uma classe de transdutor por grandeza física e os objetos SmartData que a exportam. As mensagens de interesse, resposta e comando seguem o layout de mensagens TSTP e são difundidas em UDP multicast; um nó *proxy* reconstrói as mensagens ROS a partir das respostas e recebe também SmartData de outros produtores. A validação em ambiente conteinerizado mediu uma latência de tradução de $8{,}3\,\mu\mathrm{s}$ no 95º percentil para uma varredura LiDAR de 100 feixes, uma ordem de grandeza abaixo do critério de $100\,\mu\mathrm{s}$ derivado do piso de latência do ROS 2 para cargas pequenas, entre $1\,\mathrm{ms}$ e $3\,\mathrm{ms}$; o envio das 100 respostas pela rede eleva o custo para até $2{,}5\,\mathrm{ms}$, dominado por uma chamada de sistema por SmartData. Em simulação, as mensagens recuperadas exclusivamente pela rede SmartData mantiveram as taxas originais dos sensores, e o robô foi conduzido por comandos SmartData. Os resultados sustentam a viabilidade de integrar robôs ROS 2 a ecossistemas SmartData sem comprometer o desempenho do laço de controle.

Palavras-chave: ROS 2; SmartData; Sistemas Ciber-Físicos; Interoperabilidade Semântica; Transdutores; UDP Multicast.

SUMÁRIO

1	**INTRODUÇÃO**	**6**
1.1	**Problema de Pesquisa**	**6**
1.2	**Hipótese**	**7**
1.3	**Justificativa**	**7**
1.4	**Objetivos**	**7**
1.4.1	Objetivo Geral	7
1.4.2	Objetivos Específicos	8
1.4.3	Escopo e Limites	8
2	**FUNDAMENTAÇÃO TEÓRICA**	**9**
2.1	**Sistemas Ciber-Físicos**	**9**
2.2	**Robot Operating System 2**	**9**
2.2.1	Modelo de Comunicação	10
2.2.2	Implementação por Camadas	10
2.2.3	Caracterização de Desempenho	10
2.2.4	Árvore de Transformações	10
2.3	**O Conceito de SmartData**	**11**
2.3.1	Estrutura do Envelope	11
2.3.2	Origem Espaço-Temporal e Validade	11
2.3.3	Transdutores Locais e Remotos	12
2.3.4	Interesses, Respostas e Comandos	12
2.3.5	Codificação em Pacotes de Rede	13
3	**TRABALHOS RELACIONADOS**	**14**
3.1	**Desempenho de Middleware Robótico**	**14**
3.2	**Interoperabilidade Semântica em IoT**	**14**
3.3	**Implementações de SmartData**	**15**
4	**DESIGN DO SISTEMA**	**17**
4.1	**Visão Geral**	**17**
4.2	**Tópicos ROS como Transdutores**	**18**
4.3	**Linguagem Declarativa de Sensores**	**19**
4.4	**Geração de Código em Tempo de Compilação**	**20**
4.5	**Dinâmica SmartData**	**21**
4.6	**Numeração de Dispositivos**	**22**
4.7	**Mensagens na Rede**	**23**

4.7.1 Formato das Mensagens 23
4.7.2 Transporte UDP Multicast 24
4.7.3 Espelho em Tópico ROS 24
4.8 O Lado Consumidor **24**

5 IMPLEMENTAÇÃO **26**
5.1 Estrutura do Pacote **26**
5.2 Codificação da Unidade SI **27**
5.3 Mensagens e Rede **27**
5.4 Objetos SmartData **28**
5.5 Caminho ROS para SmartData **28**
5.6 O Nó *Proxy* **28**
5.7 Geração de Código **29**

6 PLANO E EXECUÇÃO DE EXPERIMENTOS **30**
6.1 Ambiente Experimental **30**
6.2 Cenários **30**
6.2.1 Cenário Sintético 30
6.2.2 Cenário Gazebo 31
6.3 Métricas e Procedimentos **31**
6.3.1 Latência de Tradução 31
6.3.2 Latência de Tradução e Envio 31
6.3.3 Vazão Sustentada 31
6.3.4 Verificação Funcional 32
6.4 Resultados **32**
6.4.1 Latência 32
6.4.2 Vazão 33
6.4.3 Verificação Funcional 33
6.4.4 Síntese 33
6.5 Discussão **34**

7 CONSIDERAÇÕES FINAIS **36**
7.1 Trabalhos Futuros **36**

REFERÊNCIAS **38**

1 INTRODUÇÃO

A evolução dos Sistemas Ciber-Físicos (*Cyber-Physical Systems*, CPS) e a consolidação da Internet das Coisas transformaram a forma como máquinas interagem com o mundo físico. Como argumenta Lee (2008), a essência dos CPS reside na orquestração integrada de computação, comunicação e controle, onde a dinâmica dos processos físicos exige o mesmo rigor formal que as computações lógicas. Nesse cenário, a robótica móvel deixou de operar como sistema isolado e tornou-se um componente ativo em ambientes inteligentes, integrada a infraestruturas de instrumentação, controle e analítica que extrapolam os limites do robô.

No domínio do desenvolvimento robótico, o *Robot Operating System* (ROS) estabeleceu-se como o padrão de fato para a construção de sistemas complexos. O ROS 2 introduziu uma mudança arquitetural significativa ao substituir seu transporte próprio pelo *Data Distribution Service* (DDS), um padrão da OMG voltado a sistemas distribuídos de tempo real (MARUYAMA et al., 2016; MACENSKI et al., 2022). Paralelamente, no contexto de IoT industrial e cidades inteligentes, o conceito de *SmartData*, proposto por Fröhlich (2018), oferece uma abordagem orientada a dados que enriquece a informação bruta com metadados de semântica, localização e validade temporal, tornando cada leitura sensorial um objeto autocontido.

Apesar de coexistirem no mesmo nicho de aplicações ciber-físicas, os dois modelos de dados operam em níveis distintos de abstração. As mensagens nativas do ROS são projetadas para a eficiência do laço de controle e expõem campos numéricos ou estruturas pouco anotadas, das quais o significado semântico, como, unidade física, validade temporal e referencial espaço-temporal, é convenção do projeto, não dependente da mensagem. O SmartData, por sua vez, encapsula essas informações no próprio envelope, mas é difundido em um barramento próprio incompatível com o ferramental ROS. Serrano, Hauswirth e Soldatos (2015) caracterizam esse isolamento como um dos problemas centrais à interoperabilidade na Internet das Coisas, padrão que se reproduz na fronteira entre robótica e instrumentação ciber-física e motiva a tradução proposta neste trabalho.

1.1 PROBLEMA DE PESQUISA

A motivação deste trabalho está na ausência de um mecanismo direto e eficiente de tradução entre as mensagens operacionais do ROS 2 e o formato semanticamente enriquecido do SmartData. *Gateways* de aplicação genéricos existem, mas tipicamente introduzem latência incompatível com laços de controle robóticos ou perdem metadados durante a serialização. Formula-se, portanto, a seguinte questão de pesquisa: como

viabilizar a interoperabilidade semântica entre sistemas robóticos baseados em ROS 2 e plataformas de CPS orientadas a SmartData, mantendo os requisitos de tempo real e baixo *overhead* de processamento?

1.2 HIPÓTESE

Este trabalho conjectura que é possível desenvolver um componente operando nativamente no ecossistema ROS 2 capaz de interceptar, enriquecer e retransmitir mensagens de sensores no formato SmartData, com uma latência de *marshaling* pequena o suficiente para não afetar o funcionamento dos demais subsistemas. Assume-se que, ao realizar a tradução dentro do próprio nó ROS, aproveitando as proprias informações do ecossistema para preencher os metadados referentes de uma mensagem SmartData, elimina-se a necessidade de *brokers* externos complexos, preservando a integridade semântica dos dados.

1.3 JUSTIFICATIVA

A relevância desta pesquisa decorre da transição da arquitetura de sistemas robóticos para ecossistemas conectados e centrados em dados. A maturidade alcançada pelo ROS 2 (MACENSKI et al., 2022) resolveu problemas críticos de descoberta, escalabilidade e qualidade de serviço no transporte, mas não endereçou nativamente a compreensão semântica da informação: o *middleware* garante a entrega da mensagem, não que o receptor interprete inequivocamente a unidade física, o referencial de coordenadas ou a validade temporal do dado sem configuração prévia manual.

A lacuna semântica representa risco para a segurança e a confiabilidade de sistemas autônomos heterogêneos. Prestes et al. (2013) apontam que a falta de uma ontologia padronizada e acoplada aos dados em tempo de execução é causa frequente de falhas na composição de robôs de fabricantes ou módulos distintos. O SmartData mitiga esse risco ao acoplar inseparavelmente o valor numérico aos seus metadados de semântica, espaço e tempo (FRÖHLICH, 2018). Do ponto de vista de desempenho, executar a tradução dentro do próprio processo ROS, e não em um processo externo encadeado por mais uma travessia de serialização, preserva a viabilidade do caminho para laços de controle em tempo real (KRONAUER et al., 2021).

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Projetar, implementar e validar experimentalmente um *Gateway* semântico para ROS 2 capaz de interceptar mensagens de sensores padrão e retransmiti-las em tempo

real no formato SmartData, preservando a integridade semântica, espacial e temporal dos dados originais.

1.4.2 Objetivos Específicos

1. Definir um modelo de correspondência entre as mensagens de sensores e atuadores ROS e os transdutores do SmartData, expressível por meio de um arquivo declarativo.

2. Implementar o *Gateway* como nó nativo do ROS 2, aproveitando a árvore de transformações e o relógio do próprio ecossistema para enriquecer cada amostra com sua origem espaço-temporal.

3. Exportar cada grandeza como SmartData segundo a dinâmica de transdutores proposta por Fröhlich (2018), com modos de operação, interesses orientados a eventos e periódicos, regiões espaço-temporais, validade e comandos, em mensagens com o *layout* do TSTP difundidas em UDP multicast, oferecendo como caminho secundário um espelhamento em tópico ROS para instrumentação e depuração no próprio ecossistema.

4. Implementar o lado consumidor, um *proxy* que reconstrói as mensagens ROS a partir das respostas recebidas, converte mensagens ROS em comandos e recebe SmartData de outros produtores da rede.

5. Caracterizar empiricamente a latência e a vazão da tradução e do envio, e verificar o caminho de ponta a ponta através da rede SmartData.

1.4.3 Escopo e Limites

A pesquisa delimita-se à camada de *software* de aplicação sobre ROS 2 Humble em Linux, tratando o DDS subjacente apenas como o transporte padrão do ROS, sem propor modificações ao *middleware*. A validação restringe-se a cenários simulados (gerador sintético e Gazebo) para garantir a reprodutibilidade. Mecanismos criptográficos previstos pelo SmartData estão fora do escopo deste trabalho, assim como as mensagens de controle do TSTP, responsáveis pela sincronização de relógios e pela gestão de chaves, o roteamento em múltiplos saltos e a supressão de respostas redundantes prevista pelo conceito original.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os três pilares conceituais sobre os quais o *Gateway* foi construído: Sistemas Ciber-Físicos como contexto da demanda por dados semanticamente ricos, o ROS 2 como plataforma robótica baseada em DDS, e o conceito de SmartData como abstração de dados e de transdutores em redes de sensores. Cada seção descreve as definições, propriedades e limitações que motivaram decisões de projeto adotadas nos capítulos posteriores.

2.1 SISTEMAS CIBER-FÍSICOS

Sistemas Ciber-Físicos são, na definição de Lee (2008), integrações de computação e processos físicos nas quais computadores embarcados e redes monitoram e controlam o mundo físico, comumente em laços de realimentação. O autor argumenta que a passagem do tempo, no mundo físico, é inevitável e a concorrência é intrínseca; ambas as propriedades, no entanto, não são tratadas pelas abstrações usuais, herdadas da computação de propósito geral. As consequências práticas incluem a dificuldade de raciocinar formalmente sobre latência, determinismo e ordenação em sistemas que mediam controle físico.

A demanda por confiabilidade em CPS é qualitativamente distinta da encontrada em *software* convencional. Em uma aplicação de processamento de texto, uma falha pode produzir desconforto, já em um sistema de controle automotivo, ela pode produzir danos físicos. Lee enfatiza que as decisões de projeto em CPS devem incorporar explicitamente o tempo e a concorrência como cidadãos de primeira classe, e não como propriedades emergentes da implementação. Esse argumento da base à adoção de mecanismos como o *Data Distribution Service* pelo ROS 2 e ao enriquecimento temporal dos pacotes SmartData no presente trabalho.

2.2 ROBOT OPERATING SYSTEM 2

O ROS é um conjunto de bibliotecas e ferramentas para o desenvolvimento de sistemas robóticos; o ROS 2 é seu sucessor, redesenhado para abordar limitações herdadas da primeira versão em segurança, tempo real e operação em redes não ideais (MACENSKI et al., 2022). A diferença arquitetural mais relevante para este trabalho é a substituição do transporte original do ROS 1 pelo DDS, um padrão da *Object Management Group* para comunicação dirigida a dados, baseado no modelo publish/subscribe com descoberta distribuída e configurações de qualidade de serviço (QoS) (MARUYAMA et al., 2016).

2.2.1 Modelo de Comunicação

No modelo de comunicação do ROS 2, processos (*nodes*) publicam e assinam tópicos identificados por nome e tipo de mensagem. Cada tipo de mensagem é definido em uma linguagem de descrição de interface, a *Interface Definition Language* (IDL) usada pelo DDS, e compilado a estruturas de dados nativas em C++ e Python. O publicador serializa uma instância da mensagem segundo o IDL e a entrega à camada DDS, que a roteia para todos os assinantes interessados de acordo com a política de QoS associada ao tópico.

A qualidade de serviço é configurável por tópico e abrange, entre outros, profundidade do histórico, confiabilidade (*reliable* ou *best-effort*), durabilidade e prazos. O ROS 2 oferece perfis nomeados como `SensorDataQoS`, otimizado para dados sensoriais de alta frequência, e `SystemDefaultsQoS`, que herda a configuração padrão do *middleware*.

2.2.2 Implementação por Camadas

A pilha do ROS 2 segue uma estratificação clara (MACENSKI et al., 2022; MARUYAMA et al., 2016). Bibliotecas cliente como `rclcpp` expõem o modelo de *publish*/*subscribe* ao programador e delegam a comunicação a um *middleware interface module* (`rmw`), que por sua vez encapsula uma implementação concreta de DDS, como Fast DDS, Cyclone DDS ou Connext. A separação entre `rmw` e a implementação de DDS permite que o mesmo código de aplicação rode sobre fornecedores distintos.

2.2.3 Caracterização de Desempenho

A latência de comunicação do ROS 2 foi caracterizada em diversos estudos. Maruyama et al. (2016) mostraram que a sobrecarga da pilha do ROS 2 sobre o DDS bruto varia significativamente conforme o cenário, com valores da ordem de centenas de microssegundos para mensagens pequenas em processos distintos. Kronauer et al. (2021) estenderam essa análise para sistemas com múltiplos nós e mostraram que o ROS 2 pode introduzir até 50% de *overhead* sobre o DDS bruto para mensagens pequenas, com valores típicos no 95º percentil entre uma e três milissegundos para cargas de $128\,\mathrm{B}$ a $500\,\mathrm{kB}$ em configurações representativas. Esses dados são a referência mais diretamente comparável ao *overhead* adicional que um tradutor para SmartData deve evitar reintroduzir.

2.2.4 Árvore de Transformações

O subsistema `tf2` do ROS mantém uma árvore de transformações geométricas entre quadros de referência, indexada no tempo. Cada par de quadros é vinculado por uma transformação contínua, e consultas sobre essa árvore retornam, para um

instante específico, a transformação resultante da composição ao longo de um caminho da árvore. Essa estrutura é a fonte canônica para preencher os campos de origem espacial do SmartData a partir do `frame_id` declarado no cabeçalho das mensagens de sensor.

2.3 O CONCEITO DE SMARTDATA

O SmartData foi proposto por Fröhlich (2018) como a abstração primária usada por aplicações para interagir com o mundo físico em uma rede de sensores e atuadores. Trata-se de um dado enriquecido com metadados suficientes para ser autocontido quanto à semântica, à localização espacial, ao instante de produção e à confiabilidade. Conceitualmente, ler uma instância de SmartData implica amostrar o transdutor associado, e escrever em uma instância define um novo *setpoint* para um atuador.

2.3.1 Estrutura do Envelope

Cada envelope SmartData carrega, além do valor numérico, uma unidade codificada como inteiro de 32 bits que descreve a dimensão SI da grandeza sensoriada, as coordenadas (x, y, z) da origem espacial, um *timestamp* de alta resolução, um prazo de validade (*expiry*) relativo ao instante de produção e um código de autenticação. A unidade é codificada com um bit de tipo (físico/digital), dois bits para o tipo numérico, dois bits para um modificador (direto, inverso, logarítmico) e nove campos de três bits para os expoentes das sete dimensões fundamentais do SI mais radiano e esterradiano, conforme ilustrado na Figura 1 do trabalho original.

Essa codificação produz, por exemplo, a constante `METER` com expoente $+1$ na dimensão de comprimento e zero nas demais; `METER_PER_SEC_SQ` com expoentes $+1$ em comprimento e -2 em tempo, descrevendo aceleração; e `VOLT` como composto das dimensões de massa, comprimento, tempo e corrente segundo a definição formal. Portanto, a semântica é intrínseca ao identificador de unidade, sem necessidade de tabelas de tradução externas.

2.3.2 Origem Espaço-Temporal e Validade

O *timestamp* de origem é expresso em microssegundos contados a partir de uma época absoluta, por padrão, a época UNIX. A validade relativa permite que o consumidor avalie, a qualquer instante, se o dado ainda é semanticamente utilizável: um valor lido após o vencimento foi produzido em um ponto do passado em que as condições físicas observadas podem já não corresponder ao estado atual do sistema. Em Fröhlich et al. (2018), a validade é discutida como um substituto operacional para os prazos de tarefas em escalonadores de tempo real, deslocando o raciocínio temporal da execução para os dados.

2.3.3 Transdutores Locais e Remotos

A ligação entre um objeto SmartData e o mundo físico é feita por uma classe de transdutor, passada como parâmetro de tipo, que declara a constante `UNIT` segundo a codificação descrita acima (FRÖHLICH, 2018). Um SmartData cujo transdutor declara Kelvin representa a grandeza temperatura e pode abstrair tanto um sensor quanto um atuador, como um condicionador de ar. Instâncias de SmartData são criadas por um de dois construtores.

O primeiro construtor abstrai transdutores locais e recebe, além do transdutor, um identificador de dispositivo (`dev`), a validade, o modo de operação e, opcionalmente, um período. O identificador `dev` diferencia múltiplas instâncias do mesmo transdutor disponíveis em um nó. Com período diferente de zero, o transdutor é amostrado periodicamente; caso contrário, é consultado a cada acesso ao SmartData, ou sinaliza dados novos por interrupção, caso em que o acesso bloqueia até a sinalização. Três modos de operação são definidos: no modo *privado*, o SmartData pertence ao processo que o criou e não é anunciado na rede, no modo *anunciado*, ele pode ser monitorado remotamente e no modo *comandado*, além de anunciado, pode ser controlado remotamente. O modo comandado é o que distingue o conceito de outras infraestruturas de sensoriamento: em vez de enviar comandos a atuadores, a aplicação escreve um novo valor, que funciona como *setpoint*, e os atuadores da unidade correspondente localizados na região designada reagem coletivamente para alcançá-lo.

O segundo construtor cria *proxies* locais de transdutores remotos. Seu primeiro parâmetro é uma região espaço-temporal de interesse, $Region(x, y, z, r, t_0, t_f)$, em que (x, y, z) é o centro da região, r o seu raio e $[t_0, t_f]$ o intervalo de interesse; seguem-se a validade, o período e, opcionalmente, uma função de fusão. A instanciação de um SmartData remoto implica o anúncio de um interesse pela grandeza física da unidade do transdutor na região dada.

2.3.4 Interesses, Respostas e Comandos

Os SmartData que satisfazem os critérios de um interesse respondem a ele conforme o período e a função de fusão do interesse (FRÖHLICH, 2018). Período zero indica que todas as atualizações dos nós remotos devem ser reportadas, definindo uma operação orientada a eventos (*event-driven*), já outros valores exigem uma resposta a cada período, de t_0 a t_f, definindo uma operação disparada por tempo (*time-triggered*). Quando SmartData distintos usam o mesmo transdutor com períodos diferentes, a amostragem ocorre no máximo divisor comum dos períodos, embora cada objeto continue a observar o seu período original. Sem função de fusão, um SmartData responde ao interesse apenas se ainda não ouviu a resposta de outro nó, e o valor do *proxy* é o da última resposta recebida, e com função de fusão, todos os transdutores

que satisfazem o critério respondem, e cada nova resposta é entregue à função, cujo retorno define o valor do SmartData.

A validade é relativa ao instante de produção do dado e é renovada a cada atualização, assim como o período é relativo; especificar uma validade menor que o período é inválido. Nos *proxies*, a validade é uma informação do cliente: vários objetos que representam o mesmo transdutor remoto podem adotar validades distintas. Os nós que exportam SmartData anunciados gerenciam as respostas de modo a atender a todos os interesses e usam, no pacote de rede, a menor das validades envolvidas.

2.3.5 Codificação em Pacotes de Rede

A codificação dos atributos do SmartData em um pacote de rede depende do protocolo de comunicação. A implementação de referência foi construída sobre o *Trustful Space-Time Protocol* (TSTP) (RESNER et al., 2015), um protocolo *cross-layer* que provê nativamente os conceitos de tempo, localização e segurança exigidos pelo SmartData: os campos de origem, validade e autenticação são definidos e usados pelo próprio TSTP e apenas reaproveitados pela construção SmartData. Nessa implementação, o comprimento dos campos se adapta ao cenário de implantação, e campos adicionais descrevem a codificação e a versão do protocolo de cada pacote (FRÖHLICH, 2018).

3 TRABALHOS RELACIONADOS

A literatura adjacente a este trabalho organiza-se em três frentes: caracterizações de desempenho do ROS 2 e do DDS, propostas de interoperabilidade semântica para a Internet das Coisas e implementações do conceito de SmartData em pilhas de comunicação industriais. Esta seção discute cada frente e posiciona a contribuição da presente monografia em relação a elas.

3.1 DESEMPENHO DE MIDDLEWARE ROBÓTICO

Maruyama et al. (2016) produziram a primeira caracterização sistemática do desempenho do ROS 2 sobre DDS. Os autores descrevem a arquitetura em camadas, comparam fornecedores e medem latência e vazão sob configurações variáveis de QoS, tamanho de mensagem e número de processos. Sua contribuição central é demonstrar que o desempenho do ROS 2 depende fortemente do par *rmw*/DDS escolhido e da configuração de QoS, e que ganhos significativos podem ser obtidos pela seleção criteriosa desses parâmetros para o cenário de aplicação.

Kronauer et al. (2021) estenderam a análise para sistemas com múltiplos nós e quantificaram a sobrecarga adicionada pela pilha do ROS 2 em relação ao DDS bruto. Encontraram que, para mensagens pequenas (da ordem de $128\,\mathrm{B}$), o ROS 2 chega a duplicar a latência percebida em comparação com chamadas diretas ao DDS, principalmente em razão das filas internas e da indireção das interfaces `rmw`. Para mensagens grandes, a fração relativa do *overhead* encolhe e o gargalo migra para o próprio DDS.

A relevância desses resultados para o presente trabalho é direta. Um tradutor que se interponha entre a mensagem ROS original e o consumidor SmartData não pode replicar dentro do seu processo o mesmo *overhead* que a pilha já paga. As decisões de projeto descritas no Capítulo 4, subscrição tipada na própria pilha `rclcpp`, ausência de introspecção em tempo de execução e barramento UDP binário sem intermediação por DDS, são respostas diretas a essa preocupação.

3.2 INTEROPERABILIDADE SEMÂNTICA EM IOT

Serrano, Hauswirth e Soldatos (2015) discutem a fragmentação do ecossistema de Internet das Coisas em silos verticais que não interoperam e propõem uma pilha de orquestração baseada em *middleware* semântico, o OpenIoT-VDK. A estratégia dos autores é: introduz camadas de anotação, filtragem e agregação em um *broker* central que media o acesso a sensores virtuais e físicos. A abordagem é eficaz em cenários de coleta de dados em escala urbana, mas é incompatível com a latência exigida por

laços de controle robótico, pois cada amostra atravessa processos adicionais antes de chegar ao consumidor.

O presente trabalho herda a motivação, interoperabilidade semântica entre subsistemas heterogêneos, mas adota uma abordagem arquitetonicamente oposta: a tradução ocorre dentro do mesmo processo que assina o tópico ROS, sem *brokers* externos, e o consumidor SmartData recebe o envelope diretamente do barramento, sem intermediação adicional. A solução é mais restrita em escopo (não trata de agregação nem de consulta histórica), mas adequada ao domínio robótico.

3.3 IMPLEMENTAÇÕES DE SMARTDATA

O conceito de SmartData foi originalmente proposto e implementado por Fröhlich (2018) no contexto do *Embedded Parallel Operating System* (EPOS) e do *Trustful Space-Time Protocol* (TSTP), uma pilha de rede própria para sensores embarcados. A implementação original endereçava nós embarcados de baixíssimo consumo energético, com o EPOS abstraindo transdutores como objetos C++ e o TSTP provendo entrega segura e sincronizada (RESNER et al., 2015). Essa origem moldou a estrutura do envelope: a presença do campo de validade, a codificação compacta da unidade e o comprimento adaptável dos campos do pacote refletem decisões orientadas a redes de sensores com restrições severas de banda.

Em Fröhlich et al. (2018), o conceito é estendido para o projeto de CPS completos, com discussão sobre escalonamento orientado a dados, no qual tarefas são executadas para produzir os SmartData demandados antes que expirem. Essas implementações compartilham a estratégia de produzir SmartData diretamente em transdutores embarcados, abaixo de qualquer pilha de aplicação.

A implementação SDAV (LISHA/UFSC, 2026), desenvolvida no contexto do projeto de veículos autônomos do LISHA, reproduz em C++ as mensagens de interesse, resposta e comando do TSTP e as difunde em UDP multicast, levando o SmartData a computadores de propósito geral. O presente trabalho a adota como referência para o formato das mensagens, porém adaptando alguns elementos conforme necessario: o suporte de tempo de execução do *Gateway* foi escrito para as necessidades de um nó ROS, como o relógio do próprio ROS, inclusive sob tempo simulado, e a operação sobre um transporte sem garantia de entrega.

A contribuição original deste trabalho complementa essas implementações ao oferecer a ponte que faltava: a entrada do ecossistema robótico. Enquanto as implementações anteriores produziam SmartData em transdutores embarcados e os consumiam em aplicações industriais, este trabalho posiciona o gateway no ponto em que mensagens ROS já existentes precisam ser elevadas semanticamente para alcançar consumidores SmartData, sem demandar reescrita dos sensores ou do ferra-

mental robótico, e, no sentido inverso, no ponto em que SmartData produzidos na rede precisam chegar a aplicações ROS.

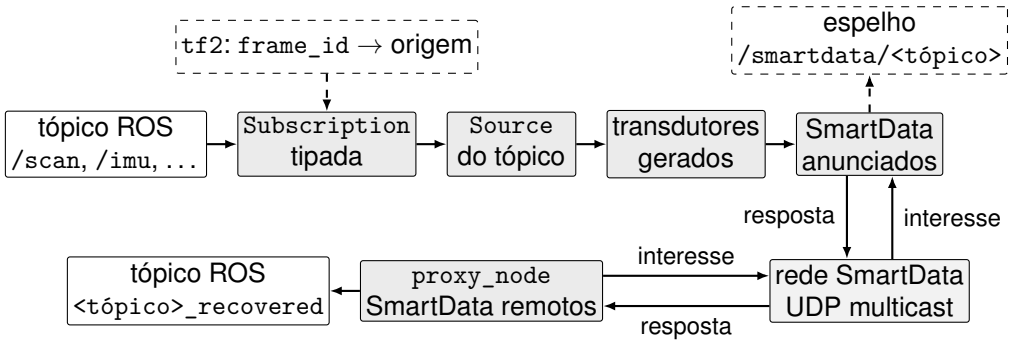
4 DESIGN DO SISTEMA

Este capítulo descreve a arquitetura do *Gateway* semântico e as decisões de projeto que conduziram à sua forma final. A apresentação parte de uma visão geral do fluxo de dados, apresenta o modelo que trata os tópicos ROS como transdutores SmartData, detalha a linguagem declarativa que descreve cada tópico, discute a estratégia de geração de código, a dinâmica SmartData adotada, a numeração de dispositivos e o formato das mensagens na rede, e termina com o lado consumidor da solução.

4.1 VISÃO GERAL

O Gateway é composto por dois executáveis ROS 2 e por um suporte de tempo de execução SmartData próprio, independente do *middleware* do ROS e baseado em *sockets* UDP multicast. O primeiro executável, `gateway_node`, exporta o robô para a rede SmartData: assina os tópicos de sensores declarados no descritor, entrega cada mensagem recebida aos transdutores que extraem dela as grandezas físicas e mantém um SmartData por instância de transdutor. Esses objetos são anunciados na rede e respondem aos interesses que recebem, os atuadores, por sua vez, recebem comandos da rede e publicam o *setpoint* resultante no tópico ROS correspondente. O segundo executável, `proxy_node`, é o lado consumidor: instancia SmartData remotos, os *proxies* do segundo construtor, que anunciam interesses, recebem as respostas e republicam as mensagens ROS reconstruídas, convertem mensagens ROS em comandos e recebem também SmartData de outros produtores da rede.

Figura 1 – Fluxo de dados entre o `gateway_node` e o `proxy_node`



Fonte: elaborada pelo autor.

A Figura 1 resume o caminho percorrido por uma mensagem. Uma mensagem ROS chega à `Subscription` tipada criada pelo gateway; o *callback* correspondente consulta a árvore `tf2` para resolver a origem espacial associada ao `frame_id` da mensagem e a entrega aos transdutores daquele tópico. Cada transdutor extrai o

seu campo, converte-o para o SI e notifica o SmartData ao qual está ligado, que responde aos interesses associados com uma mensagem de resposta difundida em UDP multicast. No `proxy_node`, as respostas que compartilham a mesma origem temporal são reagrupadas na mensagem ROS original. Opcionalmente, toda atualização local é também espelhada em um tópico ROS `/smartdata/<tópico>` do tipo `semantic_gateway/msg/SmartData`.

O suporte de tempo de execução segue a implementação SDAV do LISHA como referência de formato, mas foi adaptado para as necessidades do gateway. Três diferenças orientaram essa escolha: o relógio é injetado, para que o gateway opere sob o tempo simulado do Gazebo; a rede é um objeto, e não um estado global, para que os testes executem segmentos isolados lado a lado; e os interesses são tratados como estado leve renovado periodicamente, pois o transporte UDP não oferece a confiabilidade do TSTP (Seção 4.5).

4.2 TÓPICOS ROS COMO TRANSDUTORES

O ponto de partida do projeto é tratar cada grandeza física carregada por um tópico ROS como um transdutor no sentido de Fröhlich (2018). Uma mensagem `Imu` carrega duas grandezas, a aceleração linear, em $\mathrm{m\,s^{-2}}$, e a velocidade angular, em $\mathrm{rad\,s^{-1}}$, cada uma com três componentes. O gateway define, para ela, duas classes de transdutor, cada uma declarando a sua `UNIT`, e três instâncias de cada classe, distinguidas pelo parâmetro `dev` do primeiro construtor, que é exatamente o papel que o conceito original atribui a esse parâmetro. Analogamente, um `LaserScan` define uma única classe de comprimento, com uma instância por feixe.

Os transdutores do gateway são ativos. Não há *hardware* a amostrar, a chegada de uma mensagem no tópico assinado cumpre o papel da interrupção que sinaliza um dado novo, e cada instância extrai o seu campo, converte-o para o SI e notifica o SmartData ao qual está ligada. Diferentemente de um transdutor embarcado, cuja localização é a do próprio nó, um sensor ROS não está no nó que executa o gateway. A origem espacial de cada amostra é, por isso, resolvida na árvore `tf2` a partir do `frame_id` da mensagem, no instante indicado por `header.stamp`, e expressa em um quadro de referência configurável (`target_frame`), que faz o papel do sistema cartesiano centrado no gateway previsto pelo conceito original. Quando a transformação no instante exato ainda não está disponível, usa-se a última conhecida. A confiança da localização, de 100% para a transformação exata, 50% para a última conhecida e 0% quando nenhuma existe, é transmitida no campo correspondente da mensagem, de modo que o consumidor sabe quanto confiar na posição sem que a amostra seja descartada.

Cada instância é exportada por um SmartData do primeiro construtor, sensores

no modo anunciado e atuadores no modo comandado. Um atuador como o `/cmd_vel`, do tipo `Twist`, segue a mesma decomposição, com duas classes, velocidade linear e angular, e seis instâncias, uma por componente, cuja escrita compõe o *setpoint* publicado no tópico.

4.3 LINGUAGEM DECLARATIVA DE SENSORES

A configuração do *Gateway* é especificada em um arquivo YAML que declara, para cada tópico a ser exportado, o tipo de mensagem, o padrão de decomposição e parâmetros específicos da decomposição. Três padrões cobrem os tipos relevantes da biblioteca `sensor_msgs`:

Escalar Uma mensagem ROS carrega exatamente uma grandeza, que define uma classe de transdutor com uma instância. O campo `value_field` aponta para o atributo numérico a ser extraído, e um `value_transform` linear opcional permite ajustar a unidade (por exemplo, conversão de graus Celsius para Kelvin pela aplicação de $a \cdot v + b$). A transformação inversa é gerada automaticamente para o caminho de reconstrução.

Vetor Uma mensagem ROS carrega N valores da mesma grandeza, que definem uma classe de transdutor com uma instância por elemento de um campo vetorial. O caso prototípico é o `LaserScan`, cuja distância de cada feixe é exportada por um SmartData independente. Filtros opcionais (`finite`, `finite_within`) descartam feixes inválidos ou fora da faixa declarada do sensor; um feixe descartado não produz amostra, e o SmartData correspondente mantém o valor anterior até que ele expire.

Composto Uma mensagem ROS é decomposta em grupos de campos, cada grupo uma classe de transdutor com a sua própria unidade e uma instância por campo. O caso prototípico é o `Imu`, cujos três componentes de aceleração linear formam um grupo na unidade $\mathrm{m\,s^{-2}}$ e os três componentes de velocidade angular formam outro grupo em $\mathrm{rad\,s^{-1}}$.

Cada tópico declara também um prazo de validade (`expiry_ms`) e, opcionalmente, um período de amostragem (`period_ms`) e o modo de operação, anunciado por padrão ou privado. Uma validade menor que o período é recusada já na geração de código. A estampa de tempo de cada amostra é derivada do campo de cabeçalho ROS indicado por `stamp_field`, e a origem espacial vem da consulta `tf2` ao quadro indicado por `frame_field`. A seção `actuators` declara os atuadores com os mesmos padrões, acrescidos do quadro `tf2` que localiza o atuador (`frame`) e de uma janela de coalescência (`coalesce_ms`), discutida na Seção 4.5. A seção `imports`, apresentada

na Seção 4.8, declara SmartData produzidos fora do gateway. O Código 4.1 mostra a declaração de dois sensores.

```
sensors:
  - topic: /temperature
    type: sensor_msgs/msg/Temperature
    decomposition: scalar
    value_field: temperature
    value_transform: { kind: linear, a: 1.0, b: 273.15 }
    unit: kelvin
    expiry_ms: 500

  - topic: /imu
    type: sensor_msgs/msg/Imu
    decomposition: composite
    groups:
      - name: linear_acceleration
        unit: meter_per_sec_sq
        fields: [linear_acceleration.x, linear_acceleration.y,
                 linear_acceleration.z]
      - name: angular_velocity
        unit: rad_per_sec
        fields: [angular_velocity.x, angular_velocity.y,
                 angular_velocity.z]
    expiry_ms: 10
```

Listing 4.1 – Declaração de sensores no descritor YAML

4.4 GERAÇÃO DE CÓDIGO EM TEMPO DE COMPILAÇÃO

A tradução entre as mensagens ROS 2 e os transdutores é produzida por um gerador de código executado em tempo de compilação. O gerador, escrito em Python, lê o descritor YAML em duas etapas. Na primeira, constrói um modelo intermediário com os tópicos, as classes de transdutor e as importações declaradas, e valida a configuração: unidades desconhecidas, tópicos declarados duas vezes, validades menores que o período, transformações não inversíveis e dispositivos importados que colidem com os blocos locais interrompem a geração com uma mensagem que identifica a entrada do descritor responsável. Na segunda etapa, cada elemento do modelo é emitido como código C++.

No código emitido, cada acesso a campo aparece de forma literal (por exemplo, `msg.linear_acceleration.x`), cada `Subscription` é instanciada com o seu tipo estático e cada unidade é uma constante de compilação. O custo por mensagem reduz-se, assim, a uma sequência de acessos diretos aos campos, que o compilador pode otimizar como qualquer outro código C++. Pela mesma razão, os erros que o

gerador não detecta são detectados pelo compilador, como um campo inexistente ou um tipo incompatível no descritor resulta em erro de compilação, e não em falha durante a operação do robô. Em contrapartida, alterar o descritor exige regenerar os *headers* e recompilar os executáveis, o que é compatível com o uso pretendido, pois o descritor faz parte da configuração do robô e é alterado antes do *deploy*. O gerador emite três *headers* C++, cada um responsável por uma fatia bem delimitada do sistema:

`parsed_transducers.hpp` Contém uma classe de transdutor por grandeza de cada tópico, derivada do modelo `Transducer<UNIT>` do suporte de tempo de execução. Cada classe declara a sua unidade, o seu tipo (sensor, atuador ou ambos), o fato de ser ativa e o início do seu bloco de dispositivos. Nos sensores, o método `update` usa o deslocamento do dispositivo para escolher o seu campo, com acesso direto (por exemplo, `msg.linear_acceleration.x`), aplica a transformação e os filtros declarados e registra a amostra, já nos atuadores, o método `actuate` faz o caminho inverso.

`parsed_gateway.hpp` Contém a classe `Gateway`, usada pelo `gateway_node`, que cria as `rclcpp::Subscription<T>` tipadas, os pontos de distribuição de cada tópico aos seus transdutores, os publicadores dos atuadores e um SmartData por instância de transdutor. As instâncias dos transdutores vetoriais são criadas quando a primeira mensagem revela o número de elementos do sensor.

`parsed_proxies.hpp` Contém a classe `Proxies`, usada pelo `proxy_node`, que reúne os SmartData remotos de cada classe de transdutor, os estados de reconstrução das mensagens, cada um estritamente tipado pela mensagem alvo e com a inversa do `value_transform` embutida, as assinaturas que convertem mensagens em comandos e os SmartData importados de outros produtores.

A geração é integrada ao *build* por um `add_custom_command` do CMake que declara o descritor e o próprio gerador como dependências, de modo que os *headers* são regenerados sempre que um dos dois é modificado e, em seguida, compilados junto com os executáveis. Toda a tradução é, portanto, resolvida estaticamente, os executáveis não leem o descritor durante a execução, e a única forma da configuração presente no compilado é o código gerado.

4.5 DINÂMICA SMARTDATA

O suporte de tempo de execução do gateway implementa a dinâmica descrita na Seção 2.3.4. Um SmartData anunciado permanece em silêncio até receber um interesse compatível, com a mesma unidade, dispositivo igual ao seu ou curinga, região que contenha a origem da sua última amostra e intervalo de tempo ainda aberto.

Cada interesse aceito é guardado como uma associação, e o SmartData responde imediatamente com o seu valor corrente, para que o consumidor não precise esperar a próxima amostra, a partir daí, responde a cada atualização do transdutor, para os interesses orientados a eventos, ou a cada período, para os interesses periódicos. Interesses com períodos distintos são atendidos por um único temporizador no máximo divisor comum dos períodos. A validade enviada em cada resposta é a menor entre a do SmartData e as dos interesses associados, e um SmartData que ainda não recebeu nenhuma amostra não responde, para não difundir um valor fictício.

No TSTP, a entrega dos interesses é garantida pelo próprio protocolo. Sobre UDP, que não oferece essa garantia, um interesse perdido ou um consumidor iniciado antes do produtor quebrariam a associação de forma permanente. Adotou-se, por isso, um tratamento dos interesses como estado leve: o consumidor reanuncia cada interesse a cada segundo, e o produtor desfaz as associações não renovadas em $3{,}5\,\mathrm{s}$, o que também encerra o envio para consumidores que deixaram a rede sem revogar os seus interesses. A revogação explícita continua sendo enviada quando um *proxy* é destruído.

Os comandos seguem o caminho inverso. Escrever um valor em um SmartData remoto envia uma mensagem de comando com a região, a validade e o novo valor, os SmartData comandados da unidade correspondente cuja posição está na região repassam o valor ao transdutor, que o escreve no seu componente do *setpoint* e o reporta como nova amostra, de modo que o estado do atuador também pode ser observado na rede. Quando a mensagem ROS de um atuador tem vários campos, cada campo é exportado como um SmartData próprio, e a escrita remota de vários deles chega ao gateway como comandos independentes, um por campo. Publicar um *setpoint* a cada comando entregaria ao atuador uma rajada de mensagens parcialmente atualizadas, nas quais apenas parte dos campos já tem o novo valor. Por isso, o primeiro comando recebido abre uma janela de coalescência, de $2\,\mathrm{ms}$ por padrão e configurável por tópico no descritor: os comandos que chegam dentro dela atualizam os seus campos, os demais mantêm o último valor, e ao fim da janela uma única mensagem é publicada. Com janela nula, cada comando é publicado imediatamente, o que basta para atuadores de um único campo.

4.6 NUMERAÇÃO DE DISPOSITIVOS

A rede transporta SmartData de todos os tópicos em um único grupo multicast, e duas classes de transdutor podem compartilhar a mesma unidade, como a velocidade angular da IMU e a do `/cmd_vel`. O gerador aloca, por isso, uma faixa disjunta do espaço de dispositivos a cada classe de transdutor declarada, sequencialmente, com tamanho fixo de 2^{16} endereços. A classe de índice i recebe a faixa $[i \cdot 2^{16},\ (i+1) \cdot 2^{16})$,

e a instância de índice k recebe o dispositivo $i \cdot 2^{16} + k$. Assim, duas classes com a mesma unidade no mesmo nó nunca compartilham um dispositivo, que é o papel do parâmetro `dev` no conceito original.

Essa escolha tem duas propriedades favoráveis: primeiro, mantém as mensagens livres de qualquer campo de identificação adicional, pois o consumidor infere a mensagem e o campo de cada resposta a partir da unidade e do dispositivo, campos que já existem no SmartData; segundo, suporta sensores vetoriais com até 2^{16} elementos, suficiente para LiDARs comerciais de alta resolução. Como os interesses do *proxy* usam o dispositivo curinga, as respostas de outras classes com a mesma unidade também chegam a cada reconstrutor, que as descarta pela faixa antes de alterar a amostra em construção.

4.7 MENSAGENS NA REDE

4.7.1 Formato das Mensagens

O conceito original deixa a codificação dos atributos a cargo do protocolo (Seção 2.3.5). Adotou-se o *layout* das mensagens TSTP da implementação SDAV (LISHA/UFSC, 2026) na escala de coordenadas de 32 bits em centímetros, de modo que o deslocamento de cada campo coincide com o daquela implementação. A Tabela 1 resume as quatro estruturas. Todos os campos estão na ordem de *bytes* do *host*, *little-endian* nas plataformas consideradas, sem *padding*, o valor é um número de ponto flutuante de 64 bits, conforme o tipo numérico declarado na unidade.

Tabela 1 – Mensagens SmartData na rede.

Mensagem	**Tamanho**	**Conteúdo**
Cabeçalho comum	$60\,\mathrm{B}$	escala, tipo e versão; modo e operação; incerteza; confiança da localização; origem (x, y, z) em centímetros e t em microssegundos; unidade; dispositivo; assinatura; último salto
Interesse	$104\,\mathrm{B}$	cabeçalho; região (centro, raio, t_0, t_f); validade; período
Resposta	$78\,\mathrm{B}$	cabeçalho; validade; fragmento; valor. Sem o valor ($70\,\mathrm{B}$) nos anúncios e ocultações
Comando	$92\,\mathrm{B}$	cabeçalho, com o centro da região e t_0 na origem; raio; t_f; validade; período; valor

Fonte: elaborada pelo autor.

Os campos de assinatura são enviados zerados, pois a camada de segurança está fora do escopo, e o último salto repete a origem, pois a difusão ocorre em um único segmento de rede.

4.7.2 Transporte UDP Multicast

As mensagens são difundidas, uma por datagrama, no grupo `224.1.1.1:5000`, o mesmo da implementação de referência, com *time-to-live* igual a um e *loopback* habilitado, de modo que produtores e consumidores podem compartilhar a mesma máquina ou o mesmo processo. A escolha de multicast permite que consumidores entrem e saiam da rede sem configuração explícita do produtor. O grupo multicast é, porém, apenas o meio de difusão, fazendo com que a seleção por unidade, dispositivo e região espaço-temporal é feita pelos próprios objetos SmartData ao processar interesses e respostas, e não pela inscrição no grupo.

4.7.3 Espelho em Tópico ROS

O caminho secundário republica cada atualização dos SmartData locais em um tópico ROS por tópico de origem, `/smartdata/<tópico>`, haja ou não interesse por ela na rede, usando a mensagem `SmartData` definida no próprio pacote. Esse espelho tem propósito exclusivamente instrumental, permitindo o uso das ferramentas padrão do ecossistema ROS para inspeção (`ros2 topic echo`), gravação em *rosbag* e visualização, sem que o consumo da informação semântica dependa dele. O espelho carrega a unidade, o valor, a origem, o dispositivo e a validade de cada amostra e pode ser desativado por parâmetro (`enable_ros_mirror:=false`) quando não houver necessidade.

4.8 O LADO CONSUMIDOR

O `proxy_node` instancia, para cada classe de transdutor de sensor declarada, um SmartData remoto com dispositivo curinga, na região e no período configurados por parâmetro, de modo que um único interesse cobre, por exemplo, todos os feixes de um LiDAR. As respostas recebidas são reagrupadas pela origem temporal, que o gateway preenche com o mesmo `header.stamp` para todos os campos de uma mensagem. Sensores escalares produzem a mensagem a cada resposta, sensores compostos, quando todos os campos da amostra chegaram ou quando chega uma amostra mais nova e sensores vetoriais, quando começa uma nova varredura ou, na ausência dela, quando a varredura corrente para de receber feixes. Para os atuadores, o *proxy* mantém um SmartData remoto por instância, com o dispositivo específico do componente, e a escrita de cada campo de uma mensagem publicada em `<tópico>_command` produz o comando correspondente.

O *proxy* não se limita a recuperar o que o próprio gateway exporta, ele também recebe SmartData produzidos por outros nós da rede, como outro gateway ou um nó embarcado. Esses dados são declarados na seção `imports` do descritor, e cada entrada descreve o que pedir na rede e onde publicar o resultado. Do lado da rede,

a entrada declara a unidade, por nome ou, quando o gerador não a conhece, pelo seu código de 32 bits, e o dispositivo do produtor, ou um curinga que aceita todos os produtores daquela unidade; região, período e validade podem ser definidos por entrada. Do lado do ROS, a entrada declara o tópico, o tipo da mensagem e o campo que recebe o valor. Como os dados vêm de fora, um import não tem transdutor local nem bloco de dispositivos, o *proxy* apenas anuncia o interesse e converte as respostas.

Dois aspectos exigem cuidado porque o produtor é desconhecido. O primeiro é o tempo, a estampa da mensagem pode vir da origem da amostra ou do relógio do *proxy* no recebimento, a segunda opção sendo útil quando o produtor não compartilha a base de tempo do ROS. O segundo é a identidade do produtor, com o dispositivo curinga, cada resposta gera uma mensagem, o `frame_id` pode identificar o dispositivo de origem, e as respostas dos transdutores do próprio pacote com a mesma unidade são ignoradas por padrão, para que o import receba apenas dados externos e não repita os que o próprio gateway exporta.

5 IMPLEMENTAÇÃO

Este capítulo descreve a implementação concreta do projeto descrito no capítulo anterior. A organização do código segue a estrutura de pacotes do ROS 2 `ament`, com o pacote `semantic_gateway` compreendendo as fontes em C++, o gerador em Python, o descritor de sensores em YAML, o arquivo de mensagem do espelho ROS e os arquivos de teste e de inicialização.

5.1 ESTRUTURA DO PACOTE

A organização superficial do pacote ROS é a seguinte:

```
semantic_gateway/
  CMakeLists.txt
  package.xml
  msg/SmartData.msg
  config/sensors.yaml
  codegen/generate.py
  include/
    si_units.hpp
    smartdata/
      wire.hpp
      network.hpp
      transducer.hpp
      smartdata.hpp
      ros.hpp
  src/
    smartdata/network.cpp
    gateway_node.cpp
    proxy_node.cpp
    demo_publisher_node.cpp
  launch/
    gateway.launch.py
    gazebo_gateway.launch.py
  test/
    test_si_units.cpp
    test_smartdata_runtime.cpp
    test_gateway_pipeline.cpp
    test_bus_roundtrip.cpp
    test_imports.cpp
    imports_test.yaml
```

Listing 5.1 – Estrutura de diretórios do pacote semantic_gateway

Os *headers* sob `build/generated/`, produzidos pelo gerador a cada compilação, complementam os *headers* estáticos sob `include/` no caminho de inclusão dos executáveis. O diretório `include/smartdata/`, junto com `src/smartdata/network.cpp`, forma o suporte de tempo de execução SmartData, compilado como uma biblioteca compartilhada usada pelos dois nós e pelos testes.

5.2 CODIFICAÇÃO DA UNIDADE SI

O cabeçalho `si_units.hpp` define a estrutura de *bit fields* que codifica a unidade segundo a Figura 1 de Fröhlich (2018). Cada uma das nove dimensões físicas recebe um campo de três bits com *offset* $+4$, permitindo representar expoentes inteiros de -4 a $+3$; dois bits identificam o tipo numérico (`Int32`, `Int64`, `Float32`, `Float64`); dois bits identificam um modificador opcional (direto, inverso, logarítmico); e um bit superior diferencia grandezas físicas de dados digitais. Constantes simbólicas como `METER_PER_SEC_SQ`, `KELVIN` e `VOLT` materializam essa codificação em tempo de compilação e são consumidas diretamente pelos *headers* gerados.

5.3 MENSAGENS E REDE

As estruturas `Header`, `Interest`, `Response` e `Command`, declaradas em `wire.hpp` com `#pragma pack(push,1)`, fixam o *layout* binário das mensagens da Tabela 1. Asserções estáticas sobre o tamanho de cada estrutura impedem que mudanças acidentais no *layout* comprometam silenciosamente a interoperabilidade, e um teste confere o deslocamento de cada campo. Os datagramas recebidos são copiados para as estruturas com `memcpy`, nunca por conversão de ponteiros, e um datagrama mais curto que a estrutura esperada é descartado. O mesmo arquivo define as representações usadas pelo restante do código, `Origin` e `Region`, em metros, e concentra em um único ponto a conversão entre metros e os centímetros inteiros da rede.

A classe `Network`, implementada em `network.cpp`, abre dois *sockets*: um de envio, com *time-to-live* e *loopback* configurados, e um de recepção, associado à porta e inscrito no grupo multicast. Uma *thread* receptora lê cada datagrama, interpreta apenas o seu cabeçalho e o entrega aos objetos SmartData registrados para a unidade correspondente, a interpretação do tipo da mensagem, do dispositivo e da região fica a cargo de cada objeto. A classe `Scheduler` executa, em uma única *thread*, as tarefas periódicas do suporte de tempo de execução, as respostas a interesses periódicos e a renovação dos interesses, e descarta ativações perdidas em vez de executá-las em rajada, pois, para um sensor, interessa o valor atual, e não os anteriores. O relógio usado para as estampas de tempo é uma função passada ao construtor da `Network`.

5.4 OBJETOS SMARTDATA

O cabeçalho `smartdata.hpp` implementa os dois construtores do conceito original como dois modelos de classe. O `Responsive_SmartData<T>` possui o transdutor `T` e corresponde ao primeiro construtor: observa o transdutor, mantém as associações de interesse, responde e executa comandos. O `Interested_SmartData<UNIT>` corresponde ao segundo: anuncia e renova o interesse, filtra as respostas por unidade, dispositivo e região, aplica a função de fusão, quando houver, e envia comandos quando escrito. Ambos oferecem o operador de conversão para o valor, a validade e a espera bloqueante pela próxima atualização. O cabeçalho `transducer.hpp` define a base dos transdutores, que guarda a última leitura e notifica os seus observadores a cada amostra.

O cabeçalho `ros.hpp` faz a ligação com o ROS. O modelo `Source<Msg>` distribui cada mensagem de um tópico aos transdutores ligados a ele, e o modelo `Sink<Msg>` mantém o *setpoint* corrente de um atuador, aplica a escrita de cada componente e publica a mensagem, imediatamente ou ao fim da janela de coalescência, executada por uma *thread* própria. Como um mesmo SmartData é acessado pela *thread* do executor ROS, que entrega as mensagens, pela *thread* receptora da rede, que entrega interesses e comandos, e pela *thread* do escalonador, cada objeto protege o seu estado com uma exclusão mútua e invoca observadores e envios fora dela.

5.5 CAMINHO ROS PARA SMARTDATA

O `gateway_node` encapsula o sentido direto. Durante a construção, lê os parâmetros (`target_frame`, `enable_ros_mirror`, grupo e porta da rede), cria o *buffer* `tf2`, instancia a `Network` com o relógio do nó ROS e constrói o `Gateway` gerado, passando a função de localização e, se o espelho estiver habilitado, a função que publica cada atualização no tópico `/smartdata/<tópico>`. Cada *callback* de assinatura resolve a origem da mensagem na árvore `tf2`, com a confiança descrita na Seção 4.2, e a entrega ao `Source` do tópico, de onde ela segue aos transdutores e aos SmartData. Na destruição, o `Gateway` é destruído antes da `Network`, para que cada SmartData ainda possa anunciar a sua ocultação na rede.

5.6 O NÓ *PROXY*

O `proxy_node` é o nó simétrico ao `gateway_node`. Seus parâmetros definem a região de interesse (`region_x`, `region_y`, `region_z` e `region_radius`, sendo todo o espaço o padrão), o período dos interesses (`period_ms`, zero para interesses orientados a eventos) e o período de liberação das amostras vetoriais e compostas incompletas (`drain_period_ms`). Durante a construção, o nó instancia a `Network`, também com o reló-

gio do ROS, e a classe `Proxies` gerada, que cria os publicadores `<tópico>_recovered`, os SmartData remotos e as assinaturas `<tópico>_command`. As respostas chegam pela *thread* receptora da rede, que atualiza o estado de reconstrução e publica a mensagem completa, um *timer* periódico, de $50\,\mathrm{ms}$ por padrão, libera as amostras que pararam de receber dados. Na destruição, os *proxies* revogam os seus interesses antes do fechamento da rede.

5.7 GERAÇÃO DE CÓDIGO

O gerador `codegen/generate.py` é um *script* Python que consome o YAML via `PyYAML` e produz os três *headers* já mencionados. A produção dos textos C++ é feita com *f-strings*. As principais responsabilidades do gerador são:

1. Validar o descritor e construir um modelo intermediário de tópicos, classes de transdutor e imports; qualquer erro, como unidade desconhecida, tópico repetido, validade menor que o período ou dispositivo importado dentro de um bloco local, interrompe a compilação com uma mensagem que aponta a entrada do YAML.
2. Alocar o bloco de dispositivos de cada classe de transdutor, respeitando o esquema $i \cdot 2^{16}$ descrito na Seção 4.6.
3. Emitir as classes de transdutor, com acesso direto aos campos, aplicação do `value_transform` declarado e dos filtros dos sensores vetoriais, e a escrita do componente nos atuadores.
4. Emitir a classe `Gateway`, com as assinaturas, os publicadores e os SmartData de cada instância.
5. Emitir a classe `Proxies`, com os SmartData remotos, a transformação inversa embutida, a lógica de agrupamento pela origem temporal, o descarte prévio de dispositivos de outras classes, o redimensionamento de vetores quando o índice ultrapassa a capacidade atual e os imports.

6 PLANO E EXECUÇÃO DE EXPERIMENTOS

Este capítulo apresenta o plano experimental adotado, os procedimentos de medição e os resultados obtidos. As métricas avaliadas são as exigidas pela proposta, latência de tradução e vazão sustentada de mensagens, complementadas pelo custo do envio das respostas pela rede e por uma verificação funcional de ponta a ponta. As metas correspondentes são derivadas das características do próprio ecossistema em que o tradutor opera. Para a *latência*, o alvo é permanecer ao menos uma ordem de grandeza abaixo do piso de latência do transporte ROS 2. Trabalhos de caracterização mostram, no 95º percentil, valores entre um e três milissegundos para cargas pequenas em ROS 2 (KRONAUER et al., 2021; MARUYAMA et al., 2016), adota-se, portanto, como critério de sucesso do tradutor a manutenção da latência de tradução abaixo de $100\,\mu\mathrm{s}$, o que garante contribuição desprezível ao orçamento total. Para a *vazão*, o alvo é sustentar uma taxa ao menos duas ordens de grandeza superior ao somatório das taxas nominais de amostragem dos sensores declarados no descritor deste trabalho, $10\,\mathrm{Hz}$ para o LiDAR de 360 feixes, $100\,\mathrm{Hz}$ para os seis campos da IMU e $2\,\mathrm{Hz}$ para o sensor de temperatura, totalizando aproximadamente $4.200\,\mathrm{s}^{-1}$ atualizações de SmartData, o que impede que a tradução se torne o gargalo do fluxo, mesmo sob variação de configuração ou adição de sensores.

6.1 AMBIENTE EXPERIMENTAL

Todos os experimentos foram executados em um contêiner Docker baseado em Ubuntu 22.04 com ROS 2 Humble e Fast DDS (`rmw_fastrtps_cpp`) como implementação *rmw*. O *host* é uma máquina x86_64 com processador AMD Ryzen 7 9800X3D sob WSL2. O contêiner foi construído com a cadeia de *build* `colcon` em modo `Release`, com GCC 11.4, nível de otimização `-O3` e asserções desabilitadas. A escolha do ambiente conteinerizado garante a reprodutibilidade dos resultados em qualquer máquina compatível, removendo dependências de distribuição Linux específica.

6.2 CENÁRIOS

6.2.1 Cenário Sintético

O primeiro cenário usa o nó `demo_publisher_node`, que produz mensagens sintéticas com taxas e dimensões fixas: `LaserScan` com 360 feixes a $10\,\mathrm{Hz}$, `Imu` a $100\,\mathrm{Hz}$ e `Temperature` a $2\,\mathrm{Hz}$, todas as três sob uma árvore `tf2` estática. Esse cenário isola o desempenho do tradutor de flutuações do simulador físico e permite séries longas de medição em condições controladas.

6.2.2 Cenário Gazebo

O segundo cenário usa o simulador Gazebo (KOENIG et al., 2004) com o robô TurtleBot3 *burger* no mundo `turtlebot3_world`. As mensagens de sensores são produzidas pelo modelo SDF do Gazebo, com TF dinâmico acompanhando o movimento do robô, e o gateway e o *proxy* operam sob o tempo simulado. O robô é conduzido em círculo exclusivamente por comandos SmartData: um `Twist` de $0{,}10\,\mathrm{m\,s^{-1}}$ e $0{,}30\,\mathrm{rad\,s^{-1}}$ publicado em `/cmd_vel_command` é convertido pelo *proxy* em comandos na rede e aplicado pelo gateway em `/cmd_vel`. Esse cenário não substitui o sintético para fins de medição numérica precisa, pois o *jitter* introduzido pelo simulador é dominante sobre a latência do tradutor, mas demonstra a capacidade da solução de operar com um robô representativo em movimento.

6.3 MÉTRICAS E PROCEDIMENTOS

6.3.1 Latência de Tradução

A latência é medida sobre a entrega de uma mensagem `LaserScan` de 100 feixes ao `Source` do tópico, com os 100 transdutores e SmartData correspondentes instanciados, em uma série de 20.000 iterações. O intervalo medido compreende a extração de cada feixe, a aplicação dos filtros, o registro de cada amostra e a notificação de cada SmartData. A medição usa `std::chrono::steady_clock` imediatamente antes e depois da chamada, sem envolvimento de DDS nem da pilha de rede, pois nenhum interesse está associado, o objetivo é isolar o custo da tradução propriamente dita, conforme definido na proposta. O 95º percentil é relatado como métrica primária, com o teste falhando se ultrapassar o critério de aceitação de $100\,\mu\mathrm{s}$.

6.3.2 Latência de Tradução e Envio

A mesma medição é repetida, em 2.000 iterações, com um SmartData remoto interessado em todos os feixes associado aos 100 SmartData. Nessa configuração, cada mensagem produz também 100 respostas de $78\,\mathrm{B}$, cada uma enviada em seu próprio datagrama, e o intervalo medido passa a incluir a construção e o envio dessas respostas. O valor é reportado para caracterização do custo da rede, sem critério de aceitação próprio.

6.3.3 Vazão Sustentada

A vazão é medida em um *loop* fechado: a mesma mensagem de 100 feixes é entregue repetidamente ao `Source` por uma janela de $500\,\mathrm{ms}$, e o número de iterações completas é contado, nas duas configurações acima. A métrica derivada é o número de varreduras por segundo que o tradutor suporta. O critério de sucesso deriva do

agregado de entrada declarado no descritor: somando LiDAR (360 feixes $\times$ $10\,\mathrm{Hz}$), IMU (6 campos $\times$ $100\,\mathrm{Hz}$) e temperatura ($2\,\mathrm{Hz}$), obtém-se aproximadamente $4.200\,\mathrm{s}^{-1}$ atualizações de SmartData, exige-se que a vazão da tradução supere esse total em pelo menos duas ordens de grandeza, ou seja, $4{,}2 \times 10^{5}\,\mathrm{s}^{-1}$ atualizações, o que corresponde a cerca de $4.200\,\mathrm{s}^{-1}$ varreduras completas de 100 feixes, e que a vazão com envio supere o próprio agregado. Os valores efetivos são reportados para caracterização.

6.3.4 Verificação Funcional

A correção da solução é verificada por cinco suítes de testes automatizados, com 43 casos ao todo, executadas a cada compilação. A suíte `test_si_units` cobre a codificação da unidade. A suíte `test_smartdata_runtime` cobre o *layout* das mensagens e a dinâmica SmartData: o silêncio de um SmartData anunciado sem interesse e de um SmartData privado, os interesses orientados a eventos e periódicos, a menor validade na resposta, a região, o dispositivo específico, a revogação e o vencimento das associações, a recusa de validade menor que o período, os comandos, que só alcançam SmartData comandados, e a função de fusão. A suíte `test_gateway_pipeline` cobre os transdutores gerados e contém as medições de desempenho. A suíte `test_bus_roundtrip` percorre o caminho completo, de um publicador ROS ao gateway, à rede, ao *proxy* e de volta ao ROS, e de uma mensagem de comando ao `/cmd_vel`, com interesses orientados a eventos e periódicos. A suíte `test_imports` verifica os imports com produtores estrangeiros, cada um com a sua própria rede e os seus próprios dispositivos. Por fim, nos dois cenários, as taxas dos tópicos recuperados exclusivamente pela rede SmartData são comparadas às dos tópicos originais com `ros2 topic hz`.

6.4 RESULTADOS

Os valores a seguir correspondem a três execuções consecutivas das medições; quando variam entre as execuções, é relatado o intervalo observado.

6.4.1 Latência

A latência mediana de tradução foi de $6{,}6\,\mu\mathrm{s}$, com 95º percentil entre $8{,}1\,\mu\mathrm{s}$ e $8{,}3\,\mu\mathrm{s}$ para um `LaserScan` de 100 feixes, ou cerca de $80\,\mathrm{ns}$ por feixe, incluindo o registro da amostra e a notificação do SmartData. Esse valor é mais de uma ordem de grandeza inferior ao critério de aceitação derivado da literatura ($100\,\mu\mathrm{s}$, Seção 6.3) e mais de duas ordens de grandeza inferior ao piso de latência do ROS 2 para cargas pequenas reportado por Kronauer et al. (2021).

Com um consumidor interessado em todos os feixes, a mediana passou a $1{,}4\,\mathrm{ms}$ a $1{,}6\,\mathrm{ms}$, e o 95º percentil, a $2{,}2\,\mathrm{ms}$ a $2{,}5\,\mathrm{ms}$ por varredura, ou cerca de $22\,\mu\mathrm{s}$ a $25\,\mu\mathrm{s}$ por

resposta enviada. O custo, portanto, é dominado pela rede, e não pela tradução: cada SmartData é uma mensagem independente, como no TSTP, e cada mensagem custa uma chamada de sistema `sendto`, executada, neste ambiente, sobre a pilha de rede virtualizada do WSL2.

6.4.2 Vazão

O *loop* fechado mediu entre $1{,}41 \times 10^5$ e $1{,}45 \times 10^5$ varreduras por segundo de `LaserScan` de 100 feixes em um único *thread*, equivalente a cerca de $1{,}4 \times 10^7$ atualizações de SmartData por segundo, mais de trinta vezes o critério de $4{,}2 \times 10^5\,\mathrm{s}^{-1}$. Com o envio das respostas, a vazão caiu para 650 a 770 varreduras por segundo, ou $6{,}5 \times 10^4$ a $7{,}7 \times 10^4$ datagramas por segundo, entre 15 e 18 vezes o agregado de $4.200\,\mathrm{s}^{-1}$ declarado no descritor. A vazão do gateway é, assim, limitada pelo envio na rede, e não pelo custo da tradução.

6.4.3 Verificação Funcional

As 43 verificações automatizadas foram aprovadas em execuções repetidas. No cenário sintético, os tópicos recuperados pelo *proxy* exclusivamente a partir da rede SmartData apresentaram $10{,}0\,\mathrm{Hz}$ para o `LaserScan`, $99{,}8\,\mathrm{Hz}$ para o `Imu` e $2{,}0\,\mathrm{Hz}$ para o `Temperature`, as mesmas taxas dos sensores, e o gateway enviou cerca de 4.200 datagramas por segundo, exatamente o agregado declarado. Uma mensagem publicada em `/cmd_vel_command` resultou em uma única mensagem completa em `/cmd_vel`. No cenário Gazebo, o `/imu` do robô, a $199{,}7\,\mathrm{Hz}$, foi recuperado a $199{,}7\,\mathrm{Hz}$, e o `/scan`, a $5{,}0\,\mathrm{Hz}$, foi recuperado a $5{,}0\,\mathrm{Hz}$, sem mensagens incompletas, enquanto a odometria do robô confirmou a velocidade comandada pela rede SmartData, $0{,}10\,\mathrm{m\,s}^{-1}$ e $0{,}30\,\mathrm{rad\,s}^{-1}$. Em um mundo sem obstáculos ao alcance do LiDAR, todos os feixes são infinitos e descartados pelo filtro, de modo que nenhuma resposta de distância é produzida, comportamento esperado do filtro declarado.

6.4.4 Síntese

A Tabela 2 reúne as métricas medidas e as metas definidas na Seção 6.3. Todas as metas quantitativas foram atendidas com folga: a tradução ficou mais de dez vezes abaixo do limite de latência e mais de trinta vezes acima da vazão exigida, e mesmo com o envio pela rede a vazão superou em mais de quinze vezes o agregado declarado no descritor. O único custo comparável ao do próprio ROS 2 é o do envio, caracterizado aqui sem meta prévia e discutido na Seção 6.5. As taxas dos tópicos recuperados coincidiram com as dos sensores nos dois cenários.

Tabela 2 – Síntese das métricas medidas frente às metas da proposta.

Métrica	**Meta**	**Medido**
Latência de tradução (`LaserScan`, 100 feixes)	$< 100\,\mu\mathrm{s}$ (piso de $1\,\mathrm{ms}$ (KRONAUER et al., 2021) $\div$ 10)	$8{,}1\,\mu\mathrm{s}$ a $8{,}3\,\mu\mathrm{s}$ (p95)
Vazão de tradução	$> 4{,}2 \times 10^5$ atualizações/s ($100\times$ agregado do descritor)	$\sim 1{,}4 \times 10^7$ atualizações/s
Vazão com envio	> 4.200 datagramas/s (agregado do descritor)	$6{,}5$–$7{,}7 \times 10^4$ datagramas/s
Taxas recuperadas pela rede	iguais às dos sensores	sintético: 10,0 / 99,8 / $2{,}0\,\mathrm{Hz}$; Gazebo: 199,7 / $5{,}0\,\mathrm{Hz}$

Fonte: elaborada pelo autor.

6.5 DISCUSSÃO

Os resultados sustentam a hipótese declarada na Seção 1.2. A latência da tradução é suficientemente pequena para não afetar laços de controle robóticos, e as mensagens recuperadas exclusivamente pela rede SmartData preservam as taxas dos sensores e os valores dos campos eleitos como semânticos. As limitações observadas durante a execução merecem alguns comentários.

Primeiro, o custo do envio domina o custo total. Com um consumidor interessado em todos os feixes, uma varredura de 100 feixes custa algo entre dois e três milissegundos, valor da mesma ordem do piso de latência do próprio ROS 2. Esse custo decorre de cada SmartData ser uma mensagem própria, como no TSTP, e não da tradução, e ocorre apenas quando há interesse pelos dados, pois um SmartData sem interesse não envia nada. O agrupamento de respostas em um mesmo datagrama reduziria o número de chamadas de sistema, ao custo de se afastar do formato de referência.

Segundo, o ambiente de medição é um *host* de consumo, e a pilha de rede do WSL2 é virtualizada. Resultados em *hardware* embarcado, típico em robôs de campo, não foram avaliados; espera-se uma latência de tradução mais alta, mas a margem de uma ordem de grandeza em relação ao critério de aceitação sugere que ele seguirá sendo cumprido.

Terceiro, o cenário Gazebo confirma a operação funcional, mas suas medidas temporais refletem o simulador, não o tradutor, e foram intencionalmente excluídas da caracterização numérica.

Quarto, a implementação se afasta do conceito original em três pontos. Sem função de fusão, todo SmartData que satisfaz um interesse responde, pois a supressão de respostas de outros nós não foi implementada; com períodos distintos sobre o

mesmo transdutor, todos os interesses periódicos são respondidos no máximo divisor comum, e não cada um no seu período; e os interesses são renovados periodicamente, o que acrescenta um datagrama por segundo por interesse, mas torna a associação robusta à perda de pacotes e à ordem de inicialização dos nós.

7 CONSIDERAÇÕES FINAIS

Este trabalho apresentou o projeto, a implementação e a avaliação de um *Gateway* semântico que exporta sensores e atuadores ROS 2 como transdutores SmartData. A solução foi conduzida em três frentes. No plano conceitual, formalizou-se em uma linguagem declarativa o mapeamento entre os campos das mensagens robóticas e as classes de transdutor do SmartData, identificando três padrões de decomposição suficientes para cobrir os tipos relevantes da biblioteca `sensor_msgs`. No plano arquitetural, adotou-se a geração de código em tempo de compilação como caminho único de tradução e um suporte de tempo de execução próprio que reproduz a dinâmica de transdutores do conceito original, com modos de operação, interesses orientados a eventos e periódicos, regiões espaço-temporais, validade e comandos, em mensagens com o formato do TSTP. No plano da validação, demonstrou-se empiricamente que a latência de tradução permanece uma ordem de grandeza abaixo do critério de aceitação derivado do piso de latência do ROS 2 (KRONAUER et al., 2021), que o custo adicional do envio é dominado pela rede, e que as mensagens recuperadas exclusivamente pela rede SmartData preservam as taxas dos sensores, inclusive com um robô simulado em movimento conduzido por comandos SmartData.

A contribuição complementa as implementações existentes do conceito de SmartData, predominantemente voltadas para nós sensores embarcados, oferecendo uma ponte, nos dois sentidos, entre o ecossistema robótico e plataformas industriais, sem demandar reescrita dos sensores existentes ou intermediação por *brokers* de aplicação.

7.1 TRABALHOS FUTUROS

A continuidade natural deste trabalho organiza-se em cinco direções. Primeiro, a verificação experimental contra uma implementação independente de transdutor SmartData, por exemplo, a SDAV ou um nó EPOS sobre TSTP, fecharia o ciclo de interoperabilidade prometido pelo formato compartilhado. Segundo, a extensão do mecanismo de autenticação das mensagens, hoje preenchido com zeros, completaria a propriedade de confiabilidade originalmente prevista por Fröhlich (2018). Terceiro, a implementação da supressão de respostas na ausência de função de fusão e do atendimento de cada interesse periódico no seu próprio período aproximaria o suporte de tempo de execução do conceito original. Quarto, a avaliação do agrupamento de respostas por datagrama permitiria reduzir o custo de envio dominante. Quinto, a caracterização de desempenho em *hardware* embarcado representativo do que se encontra em robôs de campo permitiria contrastar os resultados deste trabalho com

cenários sob restrições mais severas de processador, memória e banda.

REFERÊNCIAS

FRÖHLICH, A. A. SmartData: an IoT-ready API for sensor networks. *International Journal of Sensor Networks*, v. 28, n. 3, p. 202–210, 2018. 6, 7, 8, 11, 12, 13, 15, 18, 27, 36

FRÖHLICH, A. A. et al. Data-centric cyber-physical systems design with SmartData. In: *Proceedings of the 2018 Winter Simulation Conference (WSC)*. Göteborg, Sweden: IEEE, 2018. p. 1274–1285. 11, 15

KOENIG, N. et al. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sendai, Japan: IEEE, 2004. p. 2149–2154. 31

KRONAUER, T. et al. Latency analysis of ROS2 multi-node systems. In: *ACM Conference on Embedded Networked Sensor Systems (SenSys)*. Coimbra, Portugal: ACM, 2021. 7, 10, 14, 30, 32, 34, 36

LEE, E. A. Cyber physical systems: Design challenges. In: *11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*. Orlando, FL, USA: IEEE, 2008. p. 363–369. 6, 9

LISHA/UFSC. *SDAV: implementação SmartData do projeto de veículos autônomos do LISHA*. 2026. Repositório de código-fonte. Disponível em: <https://gitlab.lisha.ufsc.br/iot/sdav>. Acesso em: 20 out. 2026. 15, 23

MACENSKI, S. et al. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, v. 7, n. 66, p. eabm6074, 2022. 6, 7, 9, 10

MARUYAMA, Y. et al. Exploring the performance of ROS2. In: *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*. Pittsburgh, PA, USA: ACM, 2016. 6, 9, 10, 14, 30

PRESTES, E. et al. Towards a core ontology for robotics and automation. *Robotics and Autonomous Systems*, v. 61, n. 11, p. 1193–1204, 2013. 7

RESNER, D. et al. Design rationale of a cross-layer, trustful space-time protocol for wireless sensor networks. In: *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. [S.l.]: IEEE, 2015. 13, 15

SERRANO, M.; HAUSWIRTH, M.; SOLDATOS, J. Defining the stack for service delivery models and interoperability in the Internet of Things: A practical case with the OpenIoT-VDK. *IEEE Journal on Selected Areas in Communications*, v. 33, n. 4, p. 676–689, 2015. 6, 14